RELATÓRIO TÉCNICO

Detecção de Bad Smells e Refatoração Segura

Este documento descreve a análise manual e automática de Bad Smells no gerador de relatórios em JavaScript, a refatoração aplicada com Extract Method e Replace Conditional with Polymorphism, e a validação pela suíte de testes usada como rede de segurança.

1. Análise de Smells

O arquivo original `src/ReportGenerator.js` concentra cabeçalho, filtro por perfil, formatação CSV/HTML e rodapé em um único método. Foram identificados pelo menos três smells clássicos de Fowler, confirmados depois pelo SonarJS.

1.1 Método longo e complexidade cognitiva alta

O método `generateReport` mistura várias responsabilidades e aninha condicionais de papel do usuário, tipo de relatório e limiar de valor. Isso eleva a complexidade cognitiva (27, com limite 5 no projeto). O código fica difícil de ler, de alterar (por exemplo, incluir JSON) e de testar em unidades menores: qualquer mudança exige exercitar o método inteiro.

1.2 Código duplicado e ramos colapsáveis

A montagem da linha CSV/HTML e o acúmulo de total se repetem nos ramos ADMIN e USER. No perfil USER há ainda um `if` aninhado (`item.value <= 500`) que o SonarJS marca como `no-collapsible-if`. Duplicação aumenta o risco de correção parcial: um formato é ajustado e o outro permanece inconsistente, o que quebra testes de regressão.

1.3 Números mágicos e efeito colateral

Os limiares 500 (filtro do usuário comum) e 1000 (destaque de prioridade do admin) aparecem literais, sem nome. Além disso, o código original mutava o item (`item.priority = true`), vazando regra de apresentação para o dado de entrada. Isso prejudica a manutenção (o significado do número não é óbvio) e os testes (o relatório passa a depender de estado mutável no array).

2. Relatório da ferramenta (ESLint + SonarJS)

A análise estática foi executada com `npx eslint src/`, usando `eslint-plugin-sonarjs` já configurado em `.eslintrc.json`. O plugin tornou mensurável o que a leitura manual já sugeria: a complexidade cognitiva 27 é um número objetivo, fácil de perder só “olhando” o arquivo. O `no-collapsible-if` apontou um aninhamento que a análise manual poderia tratar apenas como “mais um if”.

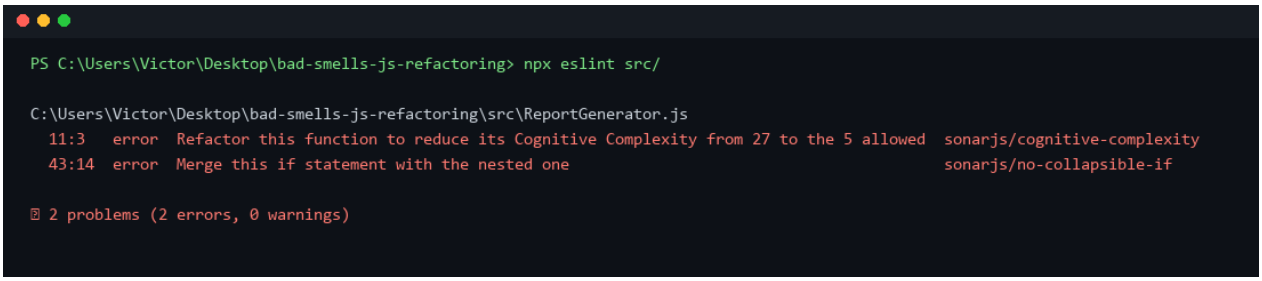

Figura 1: saída do ESLint sobre o código original (antes da refatoração).

Após a refatoração, `npx eslint src/ReportGenerator.refactored.js` não reporta erros. A suíte Jest (arquivo original e cópia apontando para o código novo) passou nos 10 testes: 5 no gerador legado e 5 no refatorado.

3. Processo de refatoração (smell mais crítico)

O smell mais crítico era a complexidade cognitiva do método único, alimentada por condicionais de formato (CSV vs HTML) entrelaçadas com regras de negócio (quem vê o quê). A estratégia foi separar filtro/totalização da renderização e substituir o "switch de formato" por polimorfismo.

Antes (trecho representativo do laço principal):

```
for (const item of items) {
  if (user.role === 'ADMIN') {
    if (item.value > 1000) { item.priority = true; }
    if (reportType === 'CSV') {
      report += `${item.id},${item.name},${item.value},${user.name}\n`;
      total += item.value;
    } else if (reportType === 'HTML') {
      const style = item.priority ? ' style="font-weight:bold;"' : '';
      report += `<tr${style}><td>${item.id}</td><td>${item.name}</td>
                 <td>${item.value}</td></tr>\n`;
      total += item.value;
    }
  } else if (user.role === 'USER') {
    if (item.value <= 500) {
      if (reportType === 'CSV') { /* mesma montagem de linha */ }
      else if (reportType === 'HTML') { /* mesma montagem de linha */ }
    }
  }
}
```

Depois (orquestração no método público):

```
generateReport(reportType, user, items) {
  const formatter = createFormatter(reportType, user);
  const visibleItems = items.filter((item) => canUserSeeItem(user, item));
  const total = sumValues(visibleItems);
  return [formatter.header(), ...visibleItems.map((item) => formatter.row(item)),
          formatter.footer(total)].join('').trim();
}
```

Técnicas aplicadas: Extract Method (`canUserSeeItem`, `sumValues`, `isAdminPriority`, `createFormatter`); Decompose Conditional para o filtro por papel; Replace Conditional with Polymorphism com `CsvReportFormatter` e `HtmlReportFormatter`, cada um responsável por cabeçalho, linha e rodapé; e Replace Magic Number with Named Constant (`USER_VALUE_LIMIT` e `ADMIN_PRIORITY_THRESHOLD`). A prioridade deixou de mutar o item e passou a ser calculada na formatação HTML.

A validação foi contínua: a suíte original não foi alterada (exceto a cópia `tests/ReportGenerator.refactored.test.js`, cujo único ajuste funcional é o import). O Jest foi habilitado a transpilar ESM via Babel para que `npm test` execute os módulos do projeto.

Observação: o HTML do admin no código inicial gerava `<tr >` (espaço extra), incompatível com a asserção da rede de segurança; o contrato dos testes foi preservado no código refatorado.

4. Conclusão

Refatorar sem testes é um palpite. Com a suíte já existente, cada extração de método pôde ser checada contra o comportamento observado (CSV/HTML, admin/usuário, lista vazia). Os testes não “aprovam o design”; eles travam o resultado. Isso permite melhorar o design com segurança.

Reduzir Bad Smells não é cosmética: o gerador passou de um procedimento de complexidade 27 para funções curtas e formatadores substituíveis. Incluir um novo formato deixa de exigir copiar blocos de if e passa a ser uma nova classe com a mesma interface. Manutenção, leitura e extensão melhoram juntas, que é exatamente o que smells como método longo e duplicação costumam impedir.